Bubble Sort !!

* ~~logic~~ = ~~You compare first & second elements, if first > second~~
  → ~~you swap them.~~

* Sorting :- It is a process of arranging items systematically !!
* Bubble sort :- It is the simplest algorithm that works by repeatedly swapping the adjacent elements if they are in wrong order.

* Example :-

First Pass :- $i = 0$

3, 1, 5, 4, 2
↓
1, 3, 5, 4, 2 (No swapping)
↓
1, 3, 5, 4, 2
↓
1, 3, 4, 5, 2
↓
1, 3, 4, 2, $\boxed{5}$

With first pass thr the entire array, the largest element (here, 5) came to the end.

Second pass :- $i = 1$

1, 3, 4, 2, 5 (Only one element will get swapped in overall 2nd pass)
↓
1, 3, 2, $\boxed{4, 5}$

With 2nd pass, second largest element comes at second from last index.

don't need to compare again & again since, this part is already sorted.

Third pass :- $i = 2$

1, 3, 2, 4, 5
↓
$\boxed{1, 2}$ $\boxed{3, 4, 5}$ sorted !!

Fourth pass :- $i = 3 \Rightarrow n-2 = 5-2 = 3$

1, $\boxed{2, 3, 4, 5}$ No swapping.

⇒ So, we require total $n-1$ ~~sw~~ passes to sort the array.
so, i goes from 0 till $n-2$ (i here is counter to count no. of passes)

→ Bubble Sort is also known as Sinking Sort or Exchange Sort.

* Let's understand more deeply how actually bubble sort works!!

Counter, $i=0$, 1st pass

j-1 j
3, 1, 5, 4, 2
0 1 2 3 4
swap ↓

1, 3, 5, 4, 2
j-1 j
swap ↓

1, 3, 4, 5, 2
j-1 j
swap ↓

1, 3, 4, 2, 5

Internal loop [it runs (n-1) times]

j ⇒ ~~[illegible]~~ till n-1 (i.e. in first pass 1 till n-1)

j will go from

→ as j-1 = -1
→ if j=0
→ invalid index

(whole loop is for j goes from 0 to n-1)

if we optimize it (don't compare ele. which are already sorted) j will go from ~~[illegible]~~ 1 till n-i-1 or n-i-1 till 1

Counter $i=1$, 2nd pass

1, 3, 4, 2, 5
j-1 j
swap ↓

1, 3, 2, 4, 5

In third pass, j will only check till this because elements after this are already sorted.

$i=2$, 3rd pass

1, 3, 2, 4, 5
j-1 j
swap ↓

1, 2, 3, 4, 5 sorted!! still it will check till index 1 in next pass. (as it doesn't know whether the array is sorted/not?

$i=3$, 4th pass.

1, 2, 3, 4, 5
j-1 j

No swapping → as array is already sorted

* Complexity :-

j ⇒ swapping happened
j ⇒ No swapping

• Space Complexity :- O(1) // constant

⇒ Since, here no extra space is required.
i.e., like copying the array etc. is not required.
hence also known as inplace sorting algorithm.

• Time Complexity :

① Best Case → Array is sorted.

i = 0
first pass

1, 2, 3, 4, 5 → only once it ran, we don't need to check it again.
j j j j

Note :- when j never swaps for particular value of i (i.e. i = 0 or 1) (i.e. i = 0 ied using first pass. or can be in the next pages as well), it means array is sorted.
∴ you can end the program.

when array will get sorted by few swapping then for particular value of i (not 0) ⇒ array is sorted

* Best case comparisons : N − 1 ⇒ N

Since, In T.C., constants are ignored, as we don't want exact time, we just want relationship i.e mathematical funct. (i.e how time grows with i/p size) i.e we want approx running time hence constants are ignored

② Worst case → sorting descending order array to ascending order.

i = 0
first pass

5, 4, 3, 2, 1
j j
↓
4, 5, 3, 2, 1
j j
↓
4, 3, 5, 2, 1
j j
↓
4, 3, 2, 5, 1
j j
↓
4, 3, 2, 1, 5 ⇒ (N − 1 swaps)

→ 4 swaps.

i = 1
2nd pass

4, 3, 2, 1 | 5 ← already sorted
j j

inner loop run only for this part.
(inner loop)

3, 4, 2, 1, 5
j-1 j
↓
3, 2, 4, 1, 5
j-1 j
3, 2, 1, 4, 5 ⇒ (N-2) swaps

Total → 3 swaps → N-2 swaps.

---

i=2, third pass

3, 2, 1, 4, 5
j-1 j
↓
2, 3, 1, 4, 5
j-1 j
↓
2, 1, 3, 4, 5

N-3 ⇒ 5-3 = 2 swaps.

---

i=3 4th pass

2, 1, 3, 4, 5
j-1 j
↓
1, 2, 3, 4, 5

⇒ N-4 ⇒ 1 ⇒ 5-4 ⇒ swaps.

i.e. no. of times outer loop runs (only here once i.e. i=0)

i.e. no. of times inner loop runs for i=0,

Total comparisons = 1(N-1) + 1(N-2) + 1(N-3) + 1(N-4)

outer loop

= $4N - \left[\frac{N \times (N+1)}{2}\right] = 4N - (1+2+3+4)$

$\left[\frac{N(N+1)}{2}\right]$

$\left[\frac{(N-1)\times(N-1)+1}{2}\right]$

= $O\left(\frac{7N+N^2}{2}\right)$

In Time Complexity, constants & less dominating terms are ignored.

Total Comparisons = $O(N^2)$

---

* Stable Sorting Algo:-

(10) (20) (20) (30) (10)

↓ after sorting.

(10) (10) (20) (20) (30)

⇒ Here after sorting, the order of indices is maintained

⇒ Order is same when value is same of indices maintained

* Unstable Sorting algorithm: if order of indices of elements present in the original array changes after sorting → Unstable Sorting Algo.

having same value

e.g. (10) (20) (20) (30) (10)

↓after sorting

(10) (10) (20) (20) (30)

Code:- Optimized bubble sort.

```
import java.util.Arrays;
public static void main (String[] args){
    int[] arr = {5,4,1,6,7};
    int n = arr.length;
  //optimized Bubble sort
  //for passes (outer for loop) → to count no. of passes.
  for (int i=0; i<n ; i++){
      boolean isSorted = true;
      for (int j= 1; j< n-i; j++){
      //compare adjacent elements & swap if out of order
       if (arr[j-1] > arr[j]){
          int temp = arr[j-1]; //fixed the in correct temp = arr[j].
          arr[j-1] = arr[j]
          arr[j] = temp
          isSorted = false; // since swap occured, it's not sorted yet, found during particular pass at particular value of i.
       }
      }
  //if no swaps occured in the inner loop, array is already sorted.
    if (isSorted){
      break;
    }
  }
  System.out.println (Arrays.toString(arr));
}}
```

Selection Sort: (largest ele. in an specified range of indexes on array)

upper index & lower index range

* **Selection sort:** Select an element & put it on its correct index.

Example:

swap

$i=0$, 4, (5) 1, 2, (3) [get the greatest ele. of the array & put it on its correct index.]

$j=0$ to $n-1$ — 0 1 2 3 4

i.e. last or second last

$\boxed{j=0 \text{ to } n-i-1}$ In general

$i=1$, (4) 3, 1, (2), $\boxed{5}$ [arr[index] ← swap (n-i-1)th index]

$j=0$ to $n-2$

index of max. element — swap — correct index of that max ele.

swap

or

$i=3$, 2 (3) (1), $\boxed{4, 5}$ [arr[max_ele_index] swap arr[n-i-1]]

$j=0$ to $n-3$

swap — correct index of that max element

$i=4$, (2) (1), $\boxed{3, 4, 5}$

$j=0$ to $n-4$

swap

$i=5$, $\boxed{1, 2, 3, 4, 5}$ → Sorted!!

~~$j=0$ to~~ stop the loop

→ Here we selected maximum element & put it on right index, we can also do vice versa i.e. select minimum element & put it on ~~right~~ left index

---

Complexity:

Total Comparisons ⇒ 4, (5) 1, 2, (3) — swap

(4) 3, 1, (2) $\boxed{5}$ — swap

Already at correct position ignore in future steps.

2, (3) (1), $\boxed{4, 5}$

(2) (1) $\boxed{3, 4, 5}$

$\boxed{1, 2, 3, 4, 5}$

no. of Comparisons.

$(n-1)$ → comparisons required to find max ele. in an array during 1st pass

$(n-2)$

$(n-3)$

⋮

1

0

Total no of comparisons $= 0+1+2+3+\ldots(n-1)$

$$= \frac{n(n+1)}{2} = \boxed{\frac{n^2-n}{2}} \Leftarrow \frac{(n-1)(n-1+1)}{2} = \frac{n^2-1-n+1}{2} \rightarrow \frac{n^2-n}{2}$$

↳ neglect constant & less dominating terms.

* Worst case → $O(n^2)$
* Best case → $O(n^2)$

⇒ It's not stable sorting algorithm.

→ It performs well on small lists (i.e. small part of arrays which is already somewhat sorted)

Code:

```
public class Main{
public static void main(String[] args){
  int[] arr={2,-32,0,78,1};
  selection(arr);
  System.out.println(Arrays.toString(arr));
}

static void selection(int[] arr){
  for(int i=0; i<arr.length; i++){
  //find the max item in the remaining array & swap
  with correct index.
  int last = arr.length - i - 1; //~~at after fi~~
  int maxIndex = getMaxIndex(arr, 0, last); //find max ele. from index start till last.
                                  start
  Swap(arr, maxIndex, last);
  }

}

static int getMaxIndex(int[] arr, int start, int end){
  int max=start;
  for(int i=start; i<=end; i++){
    if(arr[max]< arr[i]){
      max=i;
  }}
  return max;
}
```

```
static void swap (int[] arr, int first, int second){
    int temp= arr[first];
    arr[first]= arr[second];
    arr[second]= temp;
}
```

Or

```
for (int i=0; i<n; i++){
  → int max =Integer. MIN_VALUE; // Assume the smallest
    possible integer.
  → int index =0; // will store the index of the max ele.
    in unsorted part.
  → int j;
  //Find the maximum element in the unsorted part
    of array ( from 0 to n-i-1)
    for (j=0; j<n-i; j++){
      if (arr[j] > max){
          max= arr[j];
          index =j; //store the index of the maximum
                      ele.
      }
    }
  //swap the found maximum element with the last
    element of the unsorted part.
  //j is now equal to n-i, so j-1 is the last
    unsorted index (i.e. after that index, array has already
    been sorted)
      int temp=arr[j-1];
      arr[j-1]= arr[index];
      arr[index]= temp;
  }
  //print the sorted array
  System.out.println(Arrays.toString(arr));
}
}
```

Insertion Sort:

Array ⇒ 5, 3, 4, 1, 2 [partially sorting algorithm, as it sorts array upto certain index in each pass]

→ For every index, put that index element at the correct index of LHS.

(i.e) 1st pass → i = 0, → This will be sorted in 1st pass (i.e after completion of 1st pass)

[5, 3], 4, 1, 2

↓

3, 5, 4, 1, 2

2nd pass → i = 1, → this will be sorted in 2nd pass

[3, 5, 4], 1, 2

↓

3, 4, 5, 1, 2

3rd pass → i = 2 → this will be sorted in 3rd pass

[3, 4, 5, 1], 2

↓

1, 3, 4, 5, 2

4th pass → i = 3 → this will be sorted.

[1, 3, 5, 4, 2]

↓

[1, 2, 3, 4, 5] → array is sorted.

---

*To understand what every 'i' is doing:-

outer loop:-

5, 3, 4, 1, 2
0 1 2 3 4

		i	j
Sort array till index 1	pass ①	0	(i+1 till 0) in general ⇓ i.e j will go from i+1 till 0 towards LHS in every pass)
Sort array till index 2	pass ②	1	
Sort array till index 3	pass ③	2	(as j = i+1 & when i becomes 4 (i.e n-1) j would be 5 i.e IOB and hence i will go from 0 to n-2)
Sort array till index 4	pass ④	3	

i.e. 'j' will run from 0 to (n-2)

* Working:-

(terminating condⁿ) $i < (n-2)$ (outer loop)	$j > 0$ (inner loop)
0	1
1	2
2	3
3	4

0 1 2 3 4

5, 3, 4, 1, 2

3 < 5 → swap

3, 5, 4, 1, 2

3, 5, 4, 1, 2

4 < 5 → swap

3, 4, 5, 1, 2

[now, since 3 < 4 already sorted] when element j is not smaller than element (j-1) break the loop because the previous (LHS) side array is already sorted.

3, 4, 5, 1, 2

1 < 5 → swap

3, 4, 1, 5, 2

1 < 4 → swap

3, 1, 4, 5, 2

1 < 3 → swap

1, 3, 4, 5, 2

1, 3, 4, 5, 2

2 < 5 → swap

1, 3, 4, 2, 5

2 < 4 → swap

1, 3, 2, 4, 5

2 < 3 swap

(already sorted) 1, 2, 3, 4, 5 ⟹ $\boxed{1, 2, 3, 4, 5}$

→ Now, if we take $i=4$ then $j=5$ which is index out of bound.

∴, we take $\boxed{i < (n-2)}$ where n is length of an array.

Time Complexity :-

① **Best case** :- $O(n)$ [for array which is already sorted] (in ascending order)

$i=0$ 1, 2, 3, 4, 5

j j j j $\Rightarrow (n-1)$ comparisons $\Rightarrow n$

→ j goes from 0 to $n-2$ (total $n-1$)

→ j will break after each one comparison hence time complexity $O(n)$

---

② **Worst case time complexity** :-

[for array which is sorted in descending order]

$\boxed{4, 3, 2, 1, 0}$ → i will go from 0 till $n-3$

$i=0$ $i=1$ $i=2$ $i=3$

$\Rightarrow (1\times1)+(1\times2)+(1\times3)+(1\times4)$

how many times ~~inner~~ outer for loop will run?

how many times ~~outer~~ inner for loop will run?

$\Rightarrow 1(1+2+3+4)$

$\Rightarrow 1(1+2+3+\ldots+(n-1))$

$\Rightarrow \dfrac{n(n-1)}{2}$ (here, $n = n-1$)

$\Rightarrow \dfrac{(n-1)((n-1)+1)}{2}$

$\Rightarrow \dfrac{(n-1)(n)}{2}$

$\Rightarrow \dfrac{n^2-n}{2}$

$\Rightarrow n^2 \Rightarrow$ Worst case T.C $\Rightarrow O(n^2)$

(no. of times j will get swapped in 1st pass) time.

(no. of times j will get swapped in 1st pass as break condⁿ is there in insertion sort → if swapping stops loop will increament i, or stops for that particular value of i.

$(n-1)1$

$\Rightarrow 1\times1+1\times1+1\times1+1\times1+\ldots$

$(n-1)1 \Rightarrow 1+1+1+1+\ldots+(n-1)$ times

(how many times inner for loop (how many times outer for loop will run for each value of i)) for 1 pass.

(~~due to 0-based index~~ i.e. total $(n-1)$ passes will happen even if array is sorted.

* Why to use insertion sort?

1) **Adaptive:** Steps get reduced if array is already sorted)

[i.e. if arr[j-1] < arr[j] → break; array before j is already sorted it's indicating that,

[i.e. no. of swaps are reduced as compared to bubble sort]

2) **Stable Sorting Algorithm:-**

3) **used for smaller values of n:**

works good when array is partially sorted.

It takes part in hybrid sorting algorithm. (when more no. of sorting algo. run at a time, this is part of it) that hybrid sorting algo.

Code:- assume arr is given in main() & also sout(Arrays.toString(arr))

```
static void insertion(int[] arr){
    for(int i=0; i< arr.length-1; i++){
        for (int j=i+1; j>0; j--){
            if (arr[j] < arr[j-1]){
                //swap
                int temp = arr[j];
                arr[j] = arr[j-1];
                arr[j-1] = temp;
            }
            else{ //if arr[j] > arr[j-1]
                break;
            }
        }
    }
```

i.e. array ~~before~~ ~~at~~ till arr[j] is sorted; hence break & go for next pass. ~~& array till i+1, we will sort using swaps.~~

Cyclic Sort:-

→ When given numbers from range 1 → N ⇒ use cyclic sort.

e.g Given numbers in range 1 - 5

$$\{\overset{0}{3}, \overset{1}{5}, \overset{2}{2}, \overset{3}{1}, \overset{4}{4}\}$$

we while loop for cyclic sort to avoid double incrementation of i (due to for loop & due to i = i+2 ; i++ ; i++ which is wrong)

Q.] What is cyclic sort & how it works?

$$arr = \{\overset{0}{3}, \overset{1}{5}, \overset{2}{2}, \overset{3}{1}, \overset{4}{4}\}$$ → [here the numbers are jumbled, but are from 1 to 5 (i.e. in range 1 to (N) arr.length.

When the array is sorted. in that case, all the numbers are going to be at their correct indices

so, after sorting ⇒ $\{\overset{0}{1}, \overset{1}{2}, \overset{2}{3}, \overset{3}{4}, \overset{4}{5}\}$ → Here, after sorting index will become value present at that index - 1 i.e

int index = value - 1;

OR

int index = arr[i] - 1;

(value present at current index)

Check → SWAP → MOVE

* using this we are going to sort the array ⇒
* Why Index = arr[i] - 1 & not Index = arr[i]

→ bcoz index starts from 0.

Q. might be: you have given the array find the missing number.

or you've given no's from 1 to n-1, find duplicate number?

↳ if no's are from 1 to n where will you add duplicate then? hence no's (are) from 1 to n-1 ... (e.g {1, 2, 3, 5, 4, 4}

here 6 (n) can't be added in this array in case if we have to find duplicate number.

* Worst case:

e.g. arr = { 3, 5, 2, 1, 4 } → first do index = arr[i] - 1; or value - 1;

① check is 3 at the correct index, if not ~~do 3 - 1 = 2~~ (index

[i.e. if (arr[i] != arr[index]) → swap with correct index i.e. swap arr[i] & arr[index] (i.e. element at index i & element at index 'index']

⇓

② { 2, 5, 3, 1, 4 }

→ After swapping, we know that 3 is at correct portion now, but we don't know whether the other number, which got swapped with index 2 that was at the position of 3 is correct or not so, check again, arr[i] != arr[index] → ~~& don't do i++, before~~

② 2, 5, 3, 1, 4

* is 2 at correct position?
→ No, it's not → again swap it because it should be at index 1, but it's at index 0, so swap it with element present at index 1.

③ 5, 2, 3, 1, 4

→ same theory as above (i.e. present in 2nd pt)

④ 4, 2, 3, 1, 5

→ same

⑤ [1, 2, 3, 4, 5] → 1 is there which is correct) hence ~~i++ & (as at index 0~~ now, we'll check is 1 at the correct position, if it is, then move forward & so on. & we will get our answer.

⇒ we know that every unique item is only getting swapped once.

⇒ we are not swapping incrementing i, when we are swapping so that might result in more than 'n' iteration of the loop hence, In worst case

* total swaps = n-1 (if 4 ele. in array length of 5 are placed at correct indices, do we need to check is last ele. at its correct index or not, No right, hence total swaps are n-1) but still we will check whether arr[n-1] = arr[correctindex] or not but swap won't occur

* total ~~incrementations~~ = n (i++) along with intialization

⇒ & when i is sincrementing swapping is not there & vice versa hence, total number of iterations are

⇒ Total swaps + total incrementations in i along with intialization (as loop will also run for i=0)

⇒ (n-1) + (n)

⇒ 2n-1 ≈ 2n ≈ n

$\boxed{T.C \Rightarrow O(n)}$

Code:- cyclic sort:-

```
class Solution{
  public int ~~missingNum~~ static void main(String[]
    args){
    int[] arr= {3,5,2,1,4};
    sort(arr);
    System.out.println(Arrays.toString(arr));
  }
  static void sort(int[] arr){
    int j=0;
    while (i < arr.length){
```

```
        int correctIndex = arr[i] - 1;
        if (arr[i] != arr[correctIndex]){
            if swap(arr, i, correct);
        }
        else{
            i++;
        }

    }
}
static void swap(int[] arr, int first, int
                second){
        int temp = arr[first];
        arr[first] = arr[second];
        arr[second] = temp;
    }
}
```

Q] Missing no:-

* arr=[0, 1, 2], range ⇒ [0, N]
  here ⇒ [0, 3]

hence missing number 3 so return 3,

* arr = [3, 1, 0] → return 2

→ Code:-

```
class Solution{
    public int missingNumber(int[] nums){
        int n = nums.length;
        int i=0;
        //cyclic sort: place each number x (if x<n) at index
        x
        while (i<n){ → due to 0-based index
            int correctIndex = nums[i];
            //only swap if nums[i] is in [0, n-1] & not if i==n
            as arr[n] will give IOB error
            as correctIndex = arr[i] & if i = n ⇒ arr[correctIndex] ⇒ accessing
                                                  invalid index → IOBE
```

→ hence the checking here is imp as index checking is getting done in while loop i.e i<n but n element what if is ≥n → will lead to IOBE. hence check here as well.

```
if (nums[i] < n && nums[i] != nums[correctIndex]){
  //swap to put nums[i] to its correct position.
   int temp = nums[i];
   nums[i] = nums[correctIndex];
   nums[correctIndex] = temp;
//Note: don't increment i here, because the new nums[i] might also need placement

else{ i++; //[move on when current is correct or out of range (i.e. value == n)]
  }
}
//After sorting, first index i where nums[i] != i is the missing number
 placement
for (int i=0; i<n; i++){
    if (nums[i] != i){
      return i;
    }
}
//if all 0 to n-1 are present, then missing no. is n
  return n;
}
}
```

Mistakes:-

→ you used for loop & inside it's (else{i++}), which caused some (default i++ increment) indices to be skipped (due to double increment) & broke the flow ~~invariant~~ needed for cyclic sort.

→ Cyclic sort requires re-evaluating the same index after a swap, so you need a while style loop (or manually control i) instead of letting the for loop blindly move forward (i.e. double incrementing)

Code 2:- XOR-based:-

```
public class Solution{
  public int missingNumber(int[] nums){
    int n = nums.length, ans=0;
    for (int i=1; i<=n; i++){
      ans = ans^i^nums[i-1];
  } return ans; }}
```

[4, 3, 1, 2]

0^4^1^3^2^1^3^2^4 = 0

ans, nums(i-1), when i=1

0 is missing

* explanation:-

→ XOR has the property that $x \wedge x = 0$ & $x \wedge 0 = x$ & it's commutative.

- if you XOR all numbers 1 to n with all elements in nums (which contains all except the missing one shifted by index), every number that appears in both cancels out.

→ what remains is missing no::-

eg. if nums = [3, 0, 1] with, n = 3, you compute

when ① i = 1, i = 1, i = 3 ⇒

→ ans = $(1 \wedge 3) \wedge (2 \wedge 0) \wedge (3 \wedge 1)$

$= (1 \wedge 3) \wedge (2) \wedge (3 \wedge 1)$

$= 0 \wedge 2$

ans = 2 ⇒ which simplifies to missing value 2

⇒ Complexity :- T.C = O(n)
S.C = O(1), (inplace)] For both algorithms.

---

* Find All Numbers Disappeared in an Array:-

Given an array nums of n integers where nums[i] is in the range [1, n], return an array of all the integers in the range [1, n] that do not appear in nums.

* Example:-

① Input: nums = [4, 3, 2, 7, 8, 2, 3, 1]
Output: [5, 6]

② Input: nums = [1, 1]
Output: [2]

Code:-
```
class Solution{
    public List<Integer> findDisappearedNumbers(int[] nums
        int n = nums.length;
        int i=0;
        while(i<n){
            int index = nums[i] - 1;
```

```
if (nums[i] != nums[index]){
    //swap
    int temp = nums[i];
    nums[i]=nums[index];
    nums[index] = temp;
  }
 else{
    i++;
  }

}
Arraylist<Integer> result=new ArrayList<>();
for(int j=0; j< n;j++){ → i.e. index+1 that means (index+1) number is missing.
    if(num[j] != A (j+1)) {  → i.e. at num[0]=1 should be there...
        result.add (j+1);

    }

}
return result;
}
}
```

---

* Find the Duplicate number:-

e.Given an array of integers nums containing n+1 integers where each integer is in the range [1,n] inclusive. [1 to 5 → array length ⇒ 5]+1 extra space for duplicate number.

→ There is only one repeated number in nums, return this repeated number.

→ You must solve the problem without modifying the array nums & using only constant extra space.

→ [1, n] → index = nums[i]-1;

example: 1:-

i/p - nums = [1, 3, 4, 2, 2] ⇒ nums[i] == nums[index] hence i++; ~~but i≠index~~ & i == index

0 1 2 3 4

↓ i++;

[1, 3, 4, 2, 2]

[1, 4, 3, 2, 2]

as ele. present at same index will obviously same

↓

[1, 2, 3, 4, 2]

↓ i++

[1, 2, 3, 4, 2]

↓ i++

[1, 2, 3, 4, 2] here nums[i] == nums[index] & i ≠ index;

~~i.e. else part of~~ ~~if(nums[i] != nums[index]~~

~~return nums[i];~~

hence return nums[i];

code ~~Solution~~:-

```
class Solution{
  public int findDuplicate(int[] nums){
    int n=nums.length;
    int i=0;
    while(i<n){
      int index = nums[i] -1; //due to non-zero based index or range= [1, N]
      if (nums[i] != nums[index]){
        //swap
        int temp = nums[i];
        nums[i] = nums[index];
        nums[index] = temp;
      }
      else if(i != index) {  && i≠index
      //nums[i]==nums[index]; i.e. same number on both indices -> duplicate found
        return nums[i];
      }
      else // nums[i] == nums[index] but i == index;
        i.e. if index & i is same i.e. same index ⇒ same element i.e. you are comparing ele. at 1st(i) index with ele. at 1st(index) index itself then ele will obviously be same that doesn't imply at all that it's a duplicate ele. hence, move forward.

      i++;
    }
```

```
}
return -1;
}
}
```

*Find All Duplicates in an Array:- here apply normal cyclic sort as here, if you find any Duplicate you are not directly returning it, you are just adding it which leads to ignoring else part i.e i++ & as i is not incrementing, while loop will run infinitely, error → Time Limit exceeded.

code:-

```
class Solution{
public List<Integer> findDuplicates(int[] nums){
int n=nums.length;
int i=0;
while (i<n){
int index= nums[i]-1;
if(nums[i] != nums[index]){
//swap
int temp = nums[i];
nums[i]=nums[index];
nums[index] = temp;
}
else{
//nums[i] == nums[index] & i can be (!= index) or (<= index)
i++;
}
}
ArrayList<Integer> result=new ArrayList<>();
for(int j=0; j<n; j++){
if (nums[j] != j+1){
result.add(nums[j]);
}
}
return result;
}}
```

* Set Mismatch:

You have a set of integers s, which originally contains all the numbers from 1 to n. Unfortunately, due to some error, one of the numbers in s got duplicated to another number in the set, which contains results in repetition of one number & loss of another number.

→ You are given an integer array 'nums' representing the data status of this set after the error.

→ Find the number that occurs twice & the number that is missing & (i.e duplicate number) return them in the form of an array.

(index+1) → missing no. ⇒ 3

Ex:-

I/p :- nums = [1, 2, 2, 4] (0, 1, 2, 3)

duplicate no. ⇒ 2

o/p :- [2, 3]

nums[i] → no. which is duplicated

i+1 → no. which is missing

missing number = index+1

duplicate number = no. at the 'missing index.' number'

Code:

```
public class Solution {
    public static int[] misMatch(int[] nums){
    int n= nums.length.
    int i=0;
    while (i<n){ int index= nums[i]-1;
      if(nums [i] != nums [index])
      arr[i] <= arr.length &&
      int temp= nums[i];
      nums[i]= nums[index];
      nums [index] = temp;
    } else {
      i++;
    }
}
}
```

swap

(as arr[arr.length] → accessing invalid index)

but index = arr[i]-1

~~index = n-1~~

if arr[i] == n

it's valid, as

→ index = arr[i]-1

i.e.

→ index = n-1

hence we are allowing numbers <= arr.length ← arr[n-1] → valid index

if range is from 0 to n-1 then nums[i] < n as there's int index = arr[i] & not arr[i]-1

as range is from [1, N]

```
for(int j=0; j< n; j++){
    if(arr[j] != j+1){
        return new int[] {arr[index], index+1};
    }
}
new int[] {-1, -1}; // if such numbers doesn't found
}
}
```

---

*<u>Find missing positive:</u>

Given an unsorted integer array nums, return the smallest missing positive integer.

You must implement an algorithm in O(n) time & uses constant extra space.

<u>eg:-</u>

<u>i/p:-</u> nums = [1, 2, 0]

<u>o/p:-</u> 3

⇒ while sorting an array using cyclic sort, we had only +ve numbers, but now we have -ve numbers too.

⇒ but we can't swap -ve numbers on the basis of index as index can't be -ve, hence ignore -ve numbers.

⇒ & ignore numbers > n, here number == n is allowed as index = nums[i] - 1 & if nums[i] = n then index = n-1 which is valid index hence number == n is allowed (as range is from 1 to N & not 0 to N-1)

⇒ ignore -ve numbers & numbers > N

index = nums[i] - 1; can't be -ve

index can't be -ve.

⇒ as 0 is neither +ve nor -ve hence no need to consider & ignore it if it comes.

Code:-

e.g:- nums = [(-1), 1, 2, 4]

↓ i++

[-1, (1), 2, 4]

↓ ~~i++~~ swap

[1, -1, (2), 4]

↓ ~~i++~~ swap

[1, 2, -1, (4)]

↓ ~~loop break~~ i++

here

e.g. nums = [(-1), 1, 2, 4]

↓ i++

[-1, (1), 2, 4]

↓ swap

[1, (-1), 2, 4]

↓ i++

[1, -1, (2), 4]

↓ swap

[1, 2, (-1), 4]

↓ i++

[1, 2, -1, (4)]

↓ 4 is at its correct index

[1, 2, (-1), 4] → sorted array.

at index 2, number should be 3 but here no. is -1 → which is wrong hence smallest missing +ve no ⇒ 3

(if at index 0, -ve no. is there → smallest missing +ve no is 1 → i.e. start searching from 1) → case 2 if whole array is sorted [1, 2, 3, 4] → smallest missing no ⇒ arr. length + 1

code:- public class Solution {

public static int missing (int[] nums) {

int n = nums.length;

int i = 0;

```
while( i<n){
 int index = nums[i] -1; //take no. range from [1, N] &
                         not [0, N-1] as 0 is
                         not a +ve no. hence
                         taking 0 into consideration
                         won't be useful at all.

 //swap   ignore -ve
 if (arr[i] >0 && arr[i] <= arr.length && arr[i] !=
                          ignore no. > n
    arr [index]){
     int temp = arr[i];
     arr[i] = arr[index];
     arr[index] = temp;
   }
   else{
       i++;
   }
 }

 for(int j=0; j< n; j++){
    if( nums[j] != j+1){
        return j+1;
    }
 }
 return n+1; //arr. length+1;   [1,2,3]
                                 4 => arr. length+1;
}
}
```

Recursion 4th video (resume notes after pattern questions)

r = row, c = column

① Bubble sort:-

```
   0 1 2 3
3  * * * *
2  * * *
1  * *
0  *
```

- here in pattern, ~~after~~ p we used to print a star ~~when col[index]~~ until column index == row index → newline
- once its greater than row index → newline used to get start.

* here in bubble sort, instead of printing
* first we have to check whether column index < row index → true → swap
* else → do r--& c=0; //i.e. pass r-1 & c=0 in parameters } i.e when col idx > row idx so r--& c=0
* funct^n bubbleSort() will return when r==0 as after sorting 3 ele. out of 4 are sorted, hence 4th ele would have been sorted automatically so no need to check. whether arr[0] > arr[1] in last i.e. in (n-1)th pass i.e. when r==0 (3rd pass) as definitely arr[0] < arr[1] → hence no need to swap.

as next we are checking is arr[col index] > arr[col index+1] hence as col index = row index then it will check arr[col index], arr[col index+1] & as in first pass last row/col index is 3 when col index = row index = 3 → col index+1 = 4 will give IOBError → hence only check when col index < row index & not col index <= row index

e.g. ① i=3 4, 1, 2, 3 (3,0)
j=0 j+1=1

Bubble Sort

↓

i=3 1, 4, 2, 3 (3,1)

↓

i=3 1, 2, 4, 3 (3,2)
j

when r==c → r-1 or i-1 & j=0 & not r-- as it will pass r & para. & then decrement r, which will lead to infinite recursive calls → Stack Overflow error

↓

i=2 1, 2, 3, 4 (No swap when i=2 & j=0) (2,0)
j

↓

i=2 1, 2, 3, 4
j

So on.

Bubble sort code:

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args){
        int[] arr = {4, 3, 2, 1};   why arr.length-1 (see indices of row given on ~~first page~~ previous)
        bubble (arr, arr.length-1, 0);
                     ↓ r          ↓ c
        System.out.println (Arrays.toString (arr));
    }
    public static void bubble (int[] arr, int r, int c){
        if (r==0) {
            return; //where r==0 ; n-1 ele. must be sorted hence no need to compare 1st & 2nd ele. (~~which we do when r==0~~) hence, return.
        }
        if(c < r){//
            if (arr[c] > arr[c+1]){ //when adj. ele's are equal no need to swap hence we haven't written arr[c] >= arr[c+1]
            //swap
            int temp = arr[c];
            arr[c] = arr[c+1];
            arr[c+1] = temp;
            }
            bubble (arr, r, c+1); //don't do c++ ⇒ it will lead to stack overflow error.
        }
        else {
            bubble (arr, r-1, 0);
        }
    }
}
```

check

r = arr.length = 4

Recursive tree: e.g. {2, 1, 3} ⇒ no. of passes [n-1 till 1 & when r==0 return]

PE 0 1 2

main()

↓ arr. length-1 (3-1)

bubble (arr{2, 1, 3}, 2, 0)

[c < r , 0 < 2 ✓
arr[0] > arr[1] ✓ → swap]

↓ c+1

bubble ({1, 2, 3}, 2, 1)

no swap.

↓ c+1

bubble ({1, 2, 3}; 2, 2)

↓ r-1 (as c==r)

bubble ({1, 2, 3}, 1, 0)

↓

bubble ({1, 2, 3}, 1, 1)

↓ r-1 (as c==r)

return bubble ({1, 2, 3}, 0, 0)

---

*Selection Sort:-

4 passes required = arr. length

[4, 3, 2, 8, 1]

↓

[4, 3, 2, 1], 8

↓

[1, 3, 2], 4, 8

↓

[1, 2], 3, 4, 8 → [1], 2, 3, 4, 8

↓

[1, 2, 3, 4, 8] sorted

Find max from particular range (□) of array & put it at the last index of that range

→ we are going to pass max ele. index in arg. to call selection sort() funct^n recursively.

to compare current ele & max ele ~~& go to that~~ to.

⇒ c → will keep track of current ele. & range within which we are searching

⇒ r → will keep track of no. of passes (n-1) → passes requi. to sort the array (n to 1)

Sorted array

it's actually n-1 to 1 but here, we are checking max for all elements at all ranges

Selection sort code:-

```
public class Main{
    public static void main (String[] args){
    int[] arr={4, 3, 2, 1};
    selection(arr, arr.length , 0, 0);
                                  ↳(max ele. index)
                                  ; arr(0) is our max element initially in every pass.
                                  c
                                  (current ele. index)
    System.out.println(Arrays.toString(arr));
    }

public static void selection(int[] arr, int r, int c, int max){
    if (r==0){ // total no. of passes → n-1 i.e arr.length till 1 hence when r==0 → return;
                                                  or 1 to arr.length
        return;
    if (c < r){
        if (arr[c] > arr [max]){
            selection (arr, r, c+1, c);
        }
        else{
            selection (arr, r, c+1, max);
        }
    }
    else{ //swap.
        int temp = arr[max];
        arr [max] = arr[r-1];
        arr[r-1]= temp;
        selection (arr, r-1, 0, 0);
                  ↳ max=0th index → i.e. 1st ele.
                  decrement pass no.
                  current ele. index (0th index)
                  (pass will go from arr. length till 1
                  i.e. total no. of passes → 1 to arr.length → 1 to 4 i.e equal to arr.length
    }
}
}
```

0 1 2 3 4
5 * * * *
4 * * *
3 * * *
2 * *
1 *

c=r
4 5
when c==5
temp
r--
c=0

~~0 1 2 3~~
~~5 * * *~~
~~4~~
~~3~~
~~2 (r-1)~~

→ for single pass (or to find max in a range of array) i.e. from 0 till r-1 (as c<r) not c<=r

→ to swap & proceed to the next ~~single~~ pass.

Recursive tree:- {3, 1, 4} , r = arr. length = 3

P.E.

main()

↓

selection~~Sort~~({3, 1, 4}, 3, 0, 0); (c, max)

↓ max ele = ~~arr[0] = 3~~ (max argument) arr[0] = 3 arr[0] = 3

Selection ({3, 1, 4}, 3, 1, 0); // arr[1] > arr[0] (max)

↓ max ele. = arr[0] = 3 (max) → (max = 0)

Selection ({3, 1, 4}, 3, 2, 0); // arr[2] > arr[0]

↓ max = 2 (c) (index of max ele.) argument (r, c) (i.e. if condⁿ is executed ~~or~~ else c as max is passed)

Selection ({3, 1, 4}, 3, 3, 2); arr. length (in 1st pass)

↓ swap (max ele. with last index i.e. r-1 ~~(in 1st pass)~~)

Selection ({3, 1, 4}, 2, 0, 0); // arr[0] > arr[0]

↓ max = 0 (max) 1 ≯ 3

Selection ({3, 1, 4}, 2, 1, 0); // arr[1] > arr[0] ~~→ false~~ ~~true~~ false

↓ max = 0 (max)

Selection ({3, 1, 4}, 2, 2, 0);

↓ swap(if (c == r)) & do r-1 r = 2 in 2nd pass

arr[max] & arr[r-1] i.e. arr[0] & arr[1]

selection ({1, 3, 4}, 1, 0, 0); // arr[0] > arr[0] → false.

↓ ~~arr[0]~~ max = 0 (max)

selection({1, 3, 4}, 1, 1, 0);

return

↓ swap arr[max] with arr[r-1] i.e. arr[0] with arr[0] ~~(r-1 & arr-1)~~

selection({1, 3, 4}, 0, 0, 0);

↓ r == 0 (base condⁿ)

Merge Sort:

*Approach :- to sort the array in merge sort.

e.g. [8, 3, 4, 12, 5, 6]

① Divide array into 2 halves.

[8,3, 4 ; 12, 5, 6]

② sort the 2 halves separately i.e. Get the the 2 halves sorted via recursion.

[3, 4, 8] [5, 6, 12]

③ After sorting → merge them i.e. merge the 2 sorted halves/parts.

[3, 4, 8, 5, 6, 8, 12]

---

*Explanation of 3rd point:- e.g. you have 2 arrays (to visualize how 2 halves get ~~sorted~~ merged along with sorting.

arr1 = [3, 5, 9, 19, 32] (i)

arr2 = [4, 6, 8] (j)

→ ~~con~~ make new array of size [arr1.length + arr2.length]

arr = [ ]

→ take 2 pointers i & j & compare elements at indices i & j if arr1[i] < arr2[j] → put arr[i] in new array & move i forward. else put arr[j] & move j forward.

→ when

arr = [3, 4, 5, 6, 8,]

→ now j is equal to arr2.length i.e. out of bounds, in this case

arr1 = [3, 5, 9 | 19, 32] (i remaining ele.)

arr2 = [4, 6, 8] (j)

~~add~~ write remaining (for ~~arj~~ arr2) elements from arr1 from arr1[i] till arr1[arr1.length-1] at the end of the new array

arr = [3, 4, 5, 6, 8, 9, 19, 32]

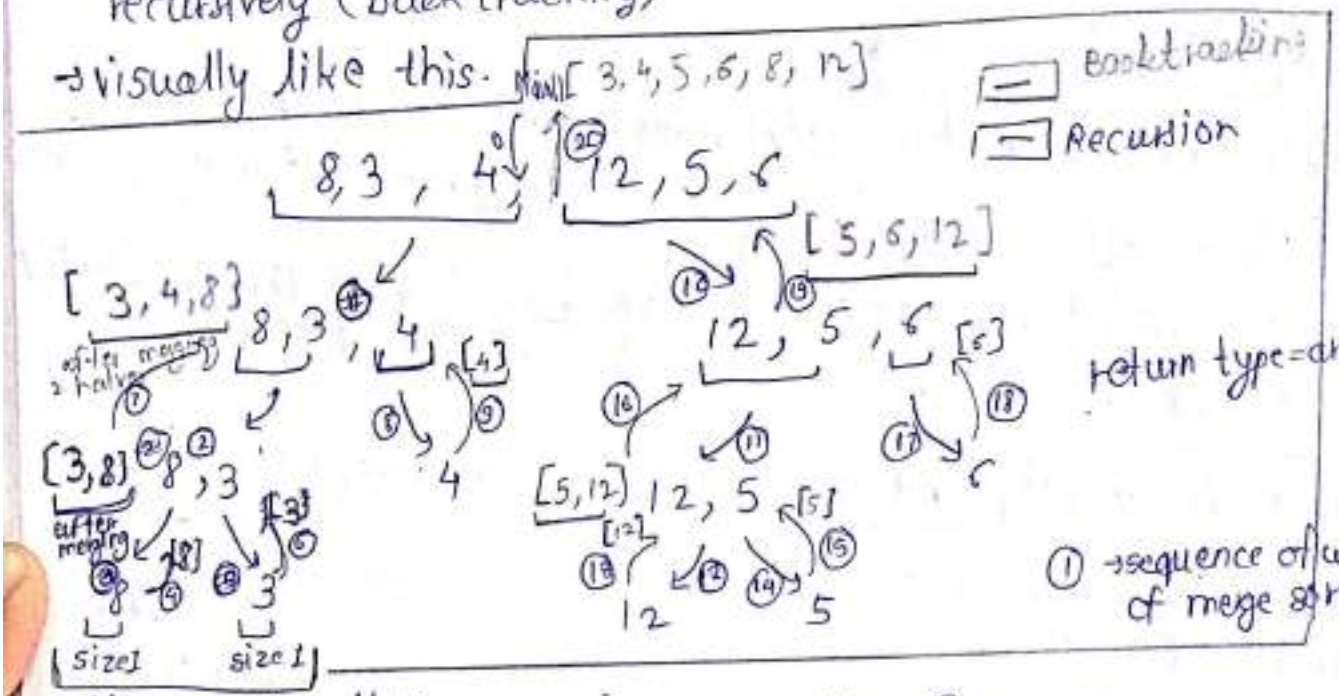
add. remaining ele. at the end of new array.

* Now do the same thing that we did in the 3rd step in recursion

→ we have to divide array into 2 halves recursively until it's size becomes 1 (recursion)

→ & then merge 2 halves of array having size 1 recursively (backtracking)

→ visually like this.



while merging these 2 sorted halves we compare them by putting i & j pointers as we did in 3rd step here, i ~~is arr[i]~~ index i is pointing to ele. 8

Divide (split into 2 halves) & Conquer (merge by comparing ith & jth ele)

(8) > (3) ⇒ arr[i] > arr[j]

→ hence after merging, array will become [3, 8] & we will return this array to previous function, in the same way we go up & get left half of the array sorted

→ then, we go to the right & get right half of the array sorted & then we merge sorted left half & right half again using comparison (i.e. i & j pointers)

Code:

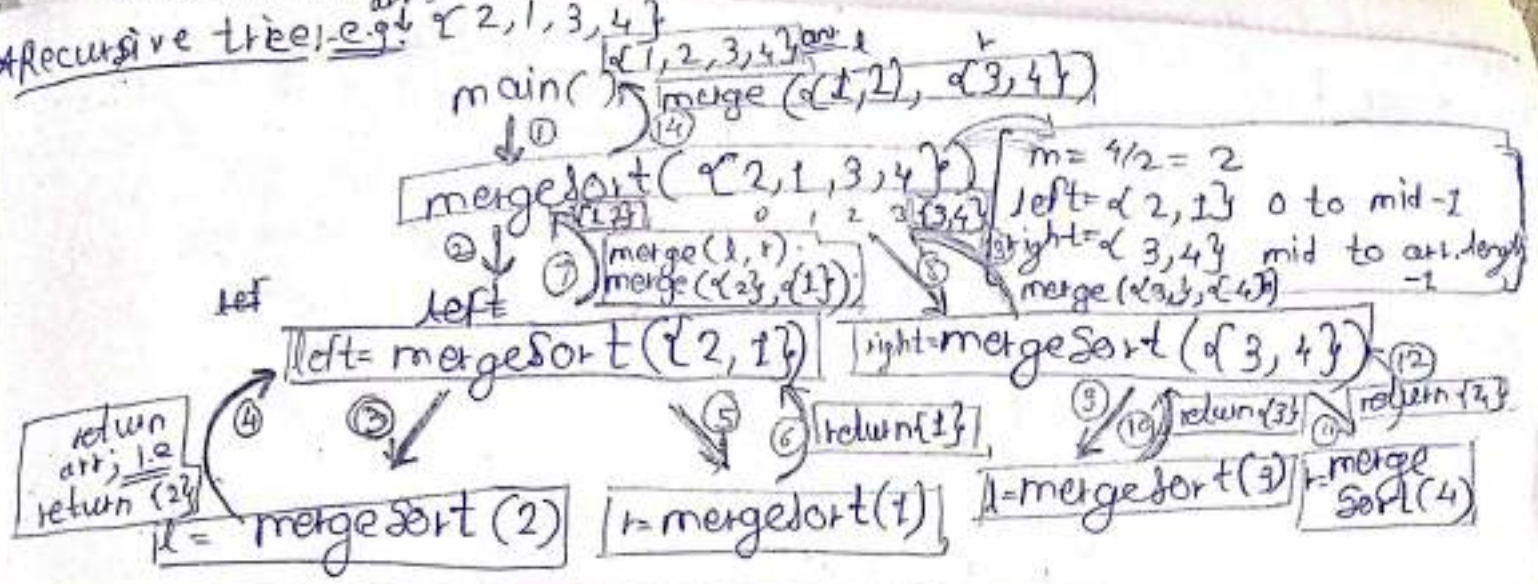
```
import java.util.Arrays;
public class Main{
    public static void main (String[] args){
    int[] arr= {3,2,1,3,4,5};
  // call mergeSort & store the sorted array.
   int [] nums = mergeSort(arr);
  //print the sorted array.
  System.out.println(Arrays.toString(nums));
}
//Recursive merge sort function//recursion→mergeSort()
public static int[] mergeSort(int[] arr){
 //Base case: if array has only one element, it's
already sorted.
 if(arr.length==1){
      return arr;// then remaining last 2 ele. will be merge/
                                  mergeSort fun^n will be
 }                                called first
                                  left, right time.
//find the middle index
int mid=arr.length/2;
//Recursively sort left half: from index 0 to mid-1
int[] left= mergeSort(Arrays.copyOfRange(arr, 0, mid));
                                  inclusive.  exclusive
//Recursively sort right: from index mid to end.
int [] right = mergeSort(Arrays.copyOfRange(arr, mid,
                        arr.length));
//Merge the sorted halves
return merge(left, right);
}
//Function to merge 2 sorted arrays.// backtracking→merge()
public static int[] merge(int[] left, int[] right){
```

```
int i=0; //pointer for left array
int j=0; //pointer for right array.
int k=0; // pointer for mix/merged array.
// Merged array size = sum of both arrays' sizes.
int[] mix = new int[] left.length + right.length];
```

*Time-Complexity

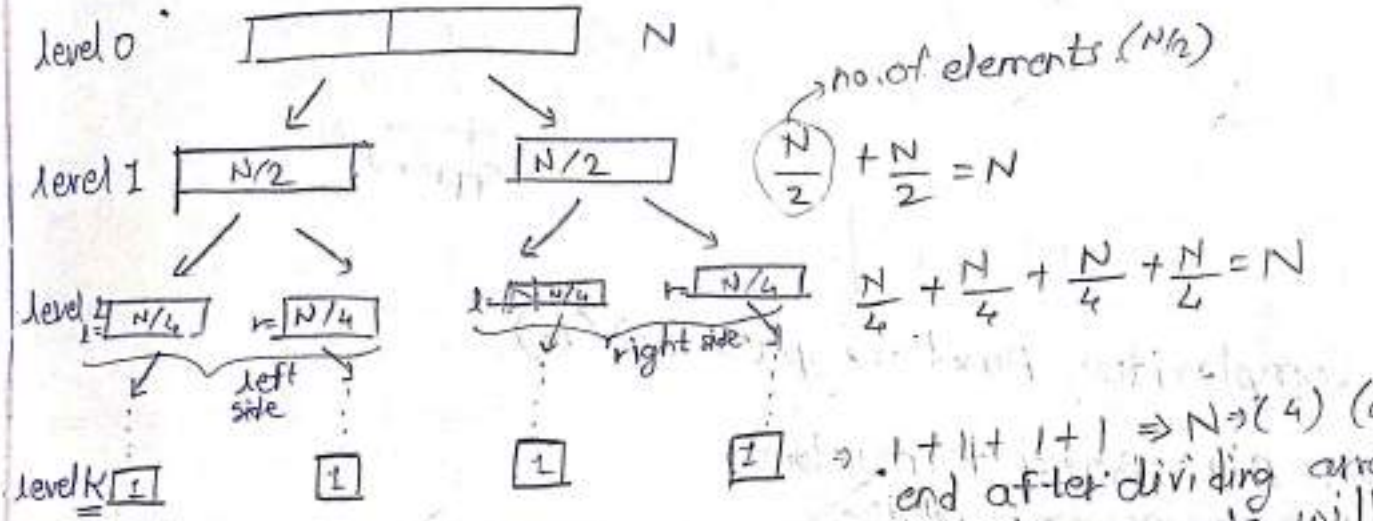
```
//Compare elements from both arrays & add smaller
 one to mix/merged array. (if any one of the ptr goes out of bound, we will come out of this while loop)
 while (i< left.length && j < right.length){
      if(left[i]< right[j]){
           mix[k]= left[i];
           i++;
      }
      else{
           mix[k]= right[j];
           j++;
      }
      k++;
 }
//Copy remaining elements from left array (if any)
 while(i< left.length){
      mix[k]= left[i];
      i++;
      k++;
 }
//copy remaining el. from right array (if any)
 while(j< right.length){
      mix[k]= right[j];
      j++;
      k++;
 }
 return mix; // return merged sorted array.
}
```

*Recursive tree:- e.g. arr = {2, 1, 3, 4}



Complexity Analysis:-

① Time Complexity:-



no. of elements (N/2)

$\frac{N}{2} + \frac{N}{2} = N$

$\frac{N}{4} + \frac{N}{4} + \frac{N}{4} + \frac{N}{4} = N$

$\Rightarrow 1 + 1 + 1 + 1 \Rightarrow N \Rightarrow (4)$ (at the end after dividing array into 2 halves one ele. will remain)

→ At every level, at the end N elements are getting/being merged (e.g at level 2 → first l=N/4 & r=N/4 elements on left are getting merged & then, l=N/4 & r=N/4 elements on right side will be merged → i.e at level 2 total N ele. will be merged → ~~& to mere~~ (we are not considering here step of dividing array which we are doing while coming down & merging we are doing while going up in recursive tree) → ~~& to merge N ele.~~ N ~~-1~~ comparisons ~~are being made~~ → ~~total comparisons~~ approx

or comparison ≤ N

↳ ~~But how?~~ → ~~will see on next page.~~

~~as one ele. takes part in exactly 1 merge.~~

* **Total No. of levels:-** Just a relation we have taken for calculating no. of level K in terms of N

$N \; (= N/2^0)$

1st level

$N/2 = N/2^1$ (no. of elements in 1 half of the array after dividing it)

2nd level

$N/4 = N/2^2$

K^{th} level

$1 = N/2^K$

$\Rightarrow$ $K+1$ levels or K levels when 1 ele. remains. ~~requ~~ will be there.

* $\frac{N}{2^K} = 1 \Rightarrow 2^K = N$

$\Rightarrow K \log 2 = \log N$

Total no. of levels $\Rightarrow K = \log_2 N$ & at each level how many comparisons are being made ~~A 1 ch~~ approx N

$$\boxed{* \; T.C \Rightarrow O(N * \log_2 N)}$$

* **Space Complexity:-** Auxillary space $\Rightarrow O(N)$

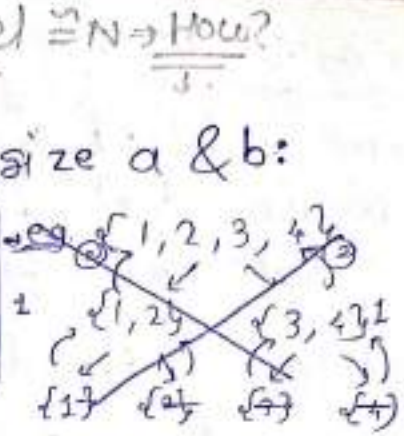
* Merge Sort - Comparisons per Level ≅ N → How?

• How comparisons are made

→ when merging 2 sorted subarrays of size a & b:

→ Maximum comparisons ⇒ a+b-1

→ last element just gets copied without comparison (as we just return last ele) ↳ without comparison



→ e.g. l={1, 2} r={3, 4} → 1 comp.
↓
{1, 2} {3, 4} ⇒ 1-comp
⇓
i will ~~go out of bound~~ {3, 4} ⇒ ~~1 comp~~ 1 incrementation & {j}=4 will be just copied to our new array.
⇓
i will go out of {1, 2, 3, 4} ⇒ total comparisons required to merge 2 sorted subarray. ⇒ ~~a+b-1~~ ⇒ ~~2+2-1=3~~ 2 (& not 3 as 3 is max comparisons required but here 1 subarray finishes early so 2 comparisons req. less than max comparisons as 2 < 3) ~~3 ≤ n~~

→ 2 comparisons.

total no. of comparison at 1st level when 2 sorted subarrays were {1, 2} & {3, 4}.

→ Hence total comparisons at each level ≤ n (to merge 2 sorted (i.e. left & right) using merge(l, r)) subarrays)
↓
as at each level we are merging 2 sorted subarrays using merge(l, r) funct.

* Why not always exactly n?

→ some merges finish early if 1 subarray is exhausted first.

→ ∴ actual comparison can be less than n at certain levels.

→ But they are never more than n - (number of merges)

*example :- (n=8)

Array = [8, 7, 6, 5, 4, 3, 2, 1]

① Level 1 (merge pairs of 1 element each → size 2):

→ merge [8] & [7] → 1 comparison
→ merge [6] & [5] → 1 comparison
→ merge [4] & [3] → 1 ⟵
→ merge [2] & [1] → 1 ⟵

{ no. of merges → 4
n = 8 (size of original array = n)

no. of comparisons ⇒ 8 - 4 ⇒ 4 ⇒ no. of comparisons ≤ 4 always.

* Total comparisons = 4 (< n = 8)

* level 2 (merge pairs of size 2 → size 4):

→ merge [7,8] & [5,6] → upto 3 comparison. (max 3 comparisons) $a+b-1$

→ merge [3,4] & [1,2] → 4 —"—

* Total comparisons = 6 ($<n=8$)

→ no. of merges → 2

$\boxed{n-2 \Rightarrow 6 \text{ (at } \leq 6)}$ (n - no. of merges at that level)

* Level 3 :- (Final merge of size 8) :-

→ merge [5,6,7,8] & [1,2,3,4] → upto to 7 comparisons (at max)

* Total comparisons = 7 ($\nless n = 8$)

[$n-1 \Rightarrow 8-1 \Rightarrow 7$ comparisons]
no. of merges
[actual compari ⇒ 7]

no. of comparison $\leq$ n - no. of merges (is true)

* Imp. pt:

① At some levels, comparisons < n (e.g., Level 1 & Level 2)

② At some levels, comparisons $\nless$ n (e.g. final merge)

③ But at no level → comparisons exceed n.

④ ∴ each level costs at max O(n) work

* Final Complexity ⇒ no. of levels = $\log_2 n$ (array keeps halving)

→ work per level $\leq n$

→ So, total comparisons:

$T(n) \leq n \log_2 n$

$T.C = O(n \cdot \log n)$

running time to sort n elements

Time complexity & not running time.

size of an array.

(1) Merge Sort Time Analysis:

① Merge-Sort Process:

m = arr. length/2; (But it is an atomic opr.)

→ We split array into 2 halves (using ↓ & copyOfRange()).

→ Then recursively sort each half (until array becomes of just 1 ele.)

→ Then merge the two sorted halves (from bottom where there is just 1 ele. upto upper part → where we have 2 sorted ~~parts~~ halves of original array → then merge 2 sorted halves of original array & return to sort main)

(2) Recurrence Relation: For an array of size n:

→ Dividing: Arrays.copyOfRange() → takes O(n) T.C (as array length is used)

→ Merging :- takes $O(n)$ → at each level (as we have seen earlier elaborately how $O(n)$ at each level?) (we create new array mix[] & combine both halves. [merging takes]

* So recurrence relation:-

$$T(n) = 2 \cdot T\left(\frac{n}{2}\right) + cn$$

→ ~~2* T(n/2) + c of each recs~~ ~~T(n/2) → time required to apply mergeSort() on both l & r halves of size n/2~~ ~~complexity~~ recursive call ↑ time used T initial of [n/2]

where:

$\circ 2 * T(n/2)$ = recursive sorting of left & right halves (half means $n/2$)

$\circ cn$ = cost of dividing (copyOfRange) + merging step.

Base case:-

$T(1) = O(1)$, (when there is only 1 ele. in an array).

③ Explaining Each Term:-

→ 2 → number of recursive calls (left & right half) $\boxed{l = \text{size} = n/2}$

```
int left = mergeSort (Arrays.copyOfRange(arr, 0,
                                mid ));//(1) recursion call
int right= mergesort (Arrays . copyOfRange(arr,
                                mid, arr.length));//(1)
                                recursion call
```

exclusive

$\boxed{r = \text{size} = n/2}$

(1) + (1) = 2

• $T(n/2)$ → time for copying merge sort() to apply on each half of size $n/2$ → initially. as $n/2$ is size of l & r halves initially.

• cn ⇒ linear time to: (non-recursive)

∘ Copy array into halves ($O(n)$) → mix[n] at each level. ↓ as we copy n elements at each level.

∘ Merge them back ($O(n)$) → at each level.

n → non-recursive bcoz it's the local merging & copying work done directly in current call & not passed down to further recursive calls. (i.e. merge() completes at that within that funct^n call of mergeSort() only & doesn't remain incomplete & goes to next funct^n call of merge(l, r)) & not incomplete funct^n of mer

④ Solving Using Master Theorem:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

• $a = 2$ (2 recursive calls)

• $b = 2$ (problem size reduced by $n/2$ at each level)

• $f(n) = cn$ [no of recursive calls per level, no of levels, dividing + merging (copying of range)]

Compare $f(n)$ with $n^{\log_b a} = n^{\log_2 2} = n^{(1)} = n$ as $\log_2 2 = 1$

Since $f(n) = \Theta(n^1)$, it matches case 2 of Master theorem.

So, $T(n) = \Theta(n \log n)$

⑤ Akra-Bazzi formula: (More general)

Akra-Bazzi handles recurrences like:

$$T(x) = g(x) + \sum_{i=1}^{k} a_i T(b_i x + h_i(x))$$

where,

- a_i = number of subproblems,
- b_i = size fraction of subproblem,
- $h_i(x)$ = small pertubation,
- $g(x)$ = non-recursive cost.

***Applying it to Merge Sort:**

$$T(n) = 2T\left(\frac{n}{2}\right) + cn$$

Here:

- $a_1 = 2$,
- $b_1 = \frac{1}{2}$,
- $h_i(x) = 0$
- $g(x) = cn$

Akra-Bazzi says:

p is the sol^n to $\sum_{i=1}^{k} a_i b_i^p = 1$

so:

- $2\left(\frac{1}{2}\right)^p = 1$
- $(1/2)^p = 1/2$
- $p = 1$

Then:-

$$T(n) = \Theta\left(n^p \left(1 + \int_1^n \frac{g(u)}{u^{p+1}} du\right)\right)$$

substitute $p=1$, $g(u) = cu$:

$$T(n) = \Theta\left(n\left(1 + \int_1^n \frac{cu}{u^2} du\right)\right)$$

$$= \Theta\left(n\left(1 + c\int_1^n \frac{1}{u} du\right)\right)$$

$$= \Theta(n(1 + c\ln n))$$

$$= \Theta(n\log n)$$

* Final Result (both Master Theorem & Akra-Bazzi):

$T(n) = \Theta(n \log n)$ → Time of merge sort.
Running

* Inplace Sorting Algorithm:-

① What we are going to do:-

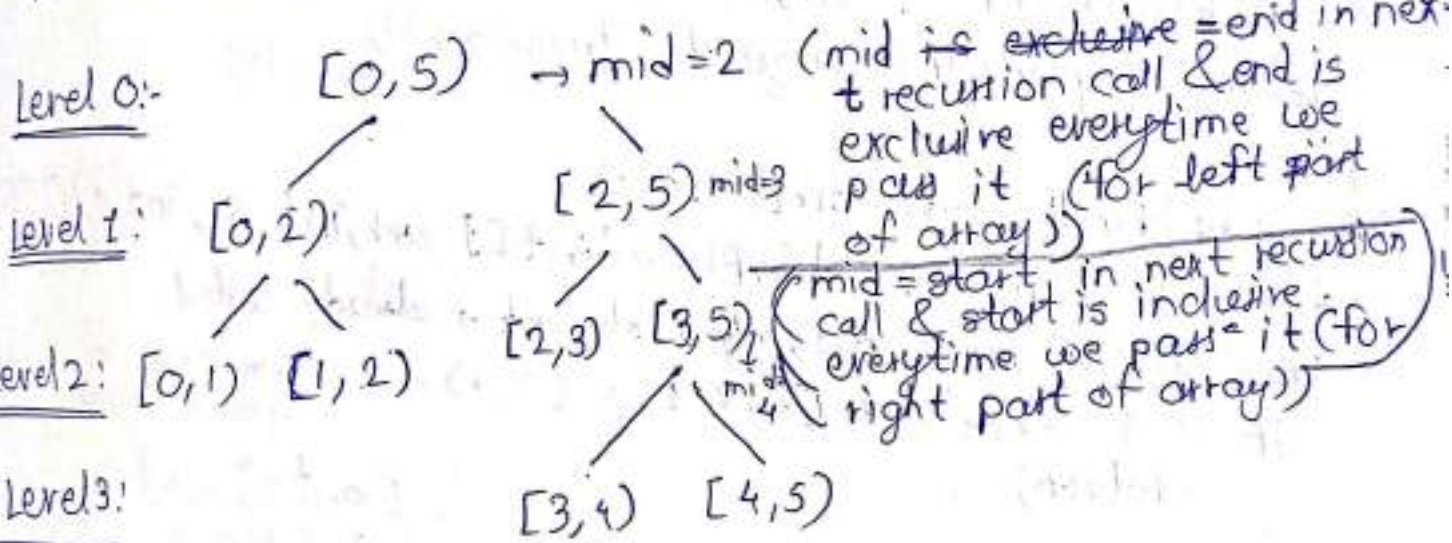
- Use Divide & Conquer:
  - Divide array into 2 halves recursively → until size=1
  - Conquer by merging 2 sorted halves.
- Work is tracked using indices [s,e) → start→inclusive, end→ exclusive.

- Recurrence:-

$T(n) = 2T(n/2) + O(n)$

② Recursion Tree for {5,4,3,2,1} (n=5):-

we call mergeSortInPlace(arr, 0, 5) → full array [0,5)



- [s,e) shows the subarray range.
- Each leaf [x, x+1) = base case (single ele → eg [3,4) → base case)

- Merging happens bottom up:

③ How we are going to do it:-

i) Step 1: Start with [0,5) (whole array).
ii) Step 2:- Split at mid = (s+e)/2
iii) Step 3:- Recursively sort [s, mid) & [mid, end)
iv) Step 4:- Merge both halves with mergeInPlace()

step 5: Merging:

→ Create temp mix[e-s]
→ Use 2 pointers: i for left [s, m), j for right [m, e)
→ Pick smaller element → put in mix array.
→ Copy leftovers if any
→ copy mix back to arr[s... e-1].

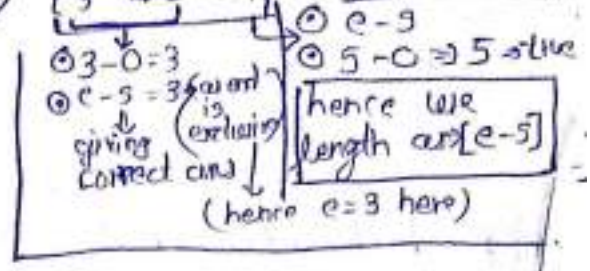
---

* Code for Inplace merge sort:

```
import java.util.Arrays;
public class MergeSort{
    public static void main(String[] args)
{ int[] arr = {5, 4, 3, 2, 1};
//sort the full array range [0, arr.length)
    mergeSortInPlace(arr, 0, arr.length);
System.out.println(Arrays.toString(arr));
}
//Recursive fun^n: sorts arr[s... e-1].
static void mergeSortInPlace(int[] arr, int s, int e){
//Base case: if range has only 1 element → already sorted
    if(e-s == 1){ // if [x, x+1) = [3,4) == 1 → Base case as size of array
        return;
    }
//Find middle index
int mid = (s+e)/2;
//recursively sort left half [s, mid)
mergeSortInPlace(arr, s, mid);
//recursively sort right half [mid, end)
mergeSortInPlace(arr, mid, e);
//merge the 2 sorted halves: ~~[s, m)~~
mergeInPlace(arr, s, mid, e);
}
```

[0, 1, 2, 3, 4]
0 1 2 3 4
[3, 4) → only 1 ele left → so return

```
//Merge 2 sorted halves: [s, m) & [m, e)
private static void mergeInPlace(int[] arr, int s, int m,
                                 int e){
    int[] mix = new int [e-s]; //
```




```
//temp array hold merged values.
    int i = s; //pter for left half
    int j = m; //pter for right half
    int k = 0; //pter fo mix array

[[3 4 5 6]]
//pick smaller ele. from left/right.
    while (i < m && j < e){
        if(arr[i] < arr[j]){
            mix[k++] = arr[i++];
        }
        else {
            mix[k++] = arr[j++];
        }
    }
//copy leftovers from left half (if any)
    while( i < m){
        mix[k++] = arr[i++];
    }
//copy leftovers from right half (if any)
    while (j < e){
        mix[k++] = arr[j++];
    }
//copy merged array back to original
    for(int l = 0; l < mix.length; l++){
        arr[s+l] = mix[l]; // s+l as we are merging part of
                              array & not array from
                              start index = 0 always.
    }
}
}
```

do dry run you'll understand

* <u>In short:</u>

i) We always work with [s, e) ranges → end exclusive.

ii) Split with mid = (s+e)/2

iii) Merge cost = O(size of subarray)

i.e [m]+[n] ⇒ [m+n]

size of merged subarray [by combining left & right halves]

iv) Each level = O(n)

v) Overall = O(n log n)

Quick Sort -

* **Pivot**: Choose any element → After first pass all the elements < Pivot will be on LHS of Pivot & all the elements > Pivot will be on RHS of Pivot.

What it is doing? (Quick sort)

① After every pass → we are putting pivot at the correct position.

i.e if arr = 5, 4, 3, 2, 1 ⟶ Pivot element

↓ After first pass

arr = $\underbrace{1, 3, 2}_{\text{all elements on LHS < Pivot}}$, 4, $\underbrace{5}_{\text{all elements on RHS > Pivot}}$

1, 3, 2 ↓ after 1 pass.

1, 2, 3 (i.e. all elements on LHS of pivot < Pivot)

~~1, 2~~

[5] already sorted ✓.

[if the array is already sorted, it won't go down or try to sort it, it'll stop there → not happening in codex]

→ [In Quick sort, I am making changes in [1, 3, 2] this part of the array in 2nd pass. [& as we are directly making change in part of original array, we don't require merge() over here → as we are not creating new array to put that part of array in it & then merging it → No → we are directly changing part of original array]]

*How to put pivot at correct position? Quick Sort Algorithm.

→ s p e

5, 4, 3, 2, 1

swap

s++
e--

s,p e

1, 4, 3, 2, 5

swap

s++;
e--;

se p

1, 2, 3, 4, 5 ⇒ ④ Now pivot is at it's correct position

1st pass.

① first we will take 3 pointers s, e & p (start, end), low & high

② while (arr[s] < pivot) → s++;

③ while (arr[e] > Pivot) → e--;

④ if (s <= e) { // & when cond^n ② & ③ both are false at the same time i.e. arr[s] ≥ Pivot && arr[e] < Pivot → swap them (arr[s] & arr[e]) → do s++ & e--; ~~it's correct position~~

~~s++, e--~~

~~se sp~~

~~1, 2, 3, 4, 5 → here s > e~~

here arr[s] < Pivot → hence acc. to cond^n while (arr[s] < Pivot) do s++;

→ but arr[e] ≯ Pivot → hence don't do anything with end

→ also when s++ will happen → s > e → i.e. cond^n violates, ~~her~~ & if we don't keep if(s <= e) cond^n in while loop after 2 while loops → swapping would have happen hence → as we kept if cond^n as if(s <= e) → then only swap → swap won't happen here & outer while loop will end (as s > e)

e s p

1, 2, 3, 4, 5 → cond^n violated → stop the outer while loop
↑ outer while loop

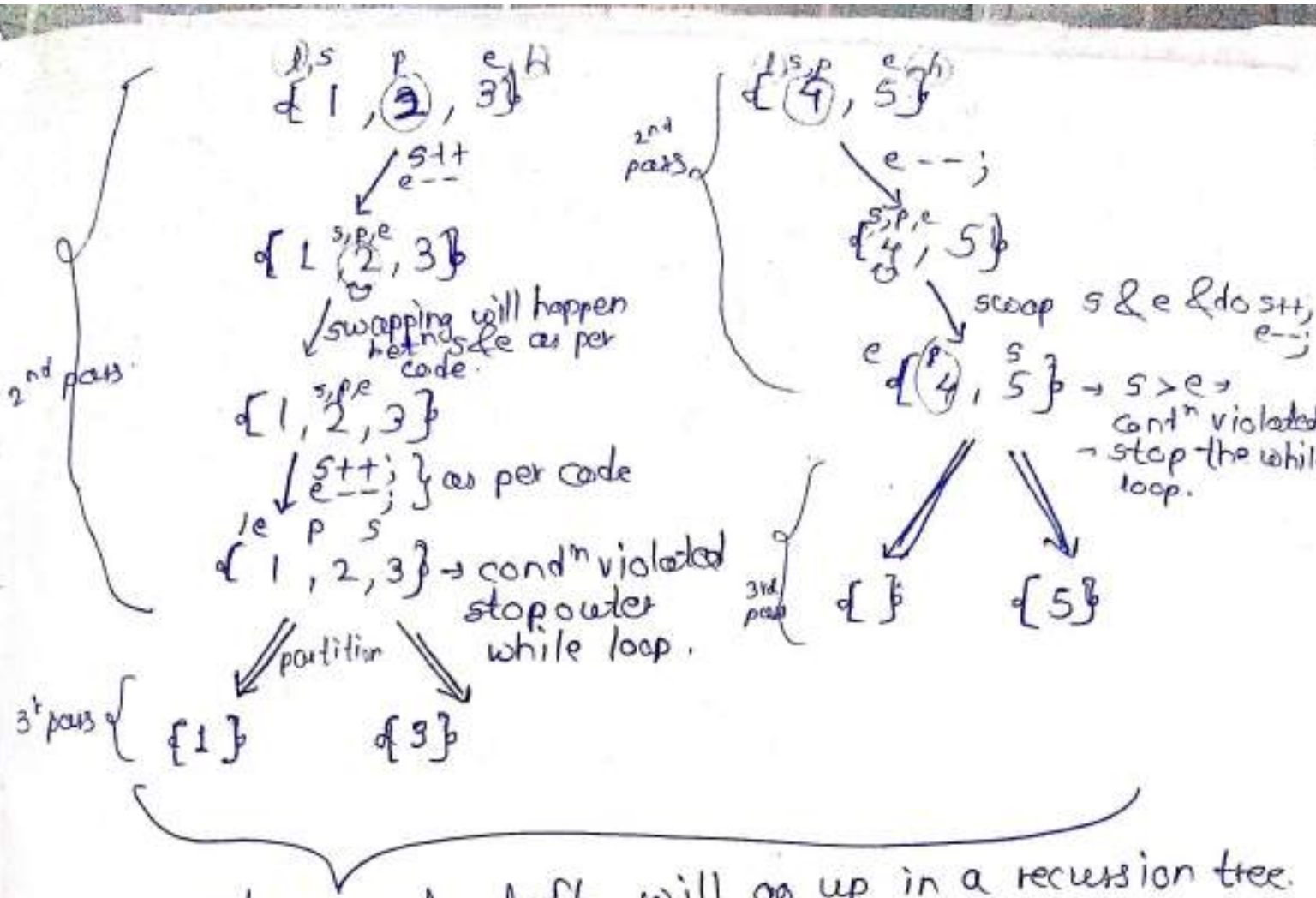
In this way in first pass pivot will come at it's correct position.

* 2nd pass → low e s high

1, 2, 3, 4, 5 → array partition will happen as
↳ left partition [low to end]
→ right partition [start to high]

[1, 2, 3] [4, 5]

→ & we will perform quick Sort again on there 2 ~~arrays~~ partitions i.e left ~~array~~ partition & right ~~array~~ partition.
not arrays



only one ele. left, will go up in a recession tree.

* s & e → just for swapping, incrementing s & decrementing e.
  low & high → main pointers tell us which part of the array you are working on
* { } → partition / part of an array. } for my understanding only (not universal notation)
* [ ] → actual array.

example ② & it's working acc. to code:

1st pass → [10, 80, 30, 90, 40, 50, 70]

↓ s++

[10, 80, 30, 90, 40, 50, 70]

↓ swap (acc. to code) & s++, e--

[10, 70, 30, 90, 40, 50, 80]

↓ s++

[10, 70, 30, 90, 40, 50, 80]

↓ swap & s++, e--

[10, 70, 30, 50, 40, 90, 80]

↓ s++

[10, 70, 30, 50, 40, 90, 80] → s > e : partition

2nd pass {10, 70, 30, 50, 40}

↓ s++

{10, 70, 30, 50, 40}

↓ swap & s++, e--

{10, 40, 30, 50, 70}

↓ s++, e--

{10, 40, 30, 50, 70} → s > e

3rd pass {10, 40, 30}

↓ s++

{10, 40, 30}

↓ swap & s++, e--

{10, 30, 40} → s > e

4th pass {10, 30} {40}

{90, 80}

↓ swap & s++, e--

{80, 90} → s > e

3rd pass {80} {90}

l == h l == h

{50, 70}

↓ s++

{50, 70}

↓ swap & s++, e--

{50, 70} → s > e

{50} {}

l == h just return - don't do anything - empty array.

when empty array is there i.e. when start > high just return. don't do anything.

s++ s,e

{10, 30}

swap & s++, e--;

s&e e h s

{10, 30}

here low = ~~start high~~ = 1 start
high = ~~start~~ = 1

return.

quickSort(arr, 2, 1) (low high) called from the body

which leads to creating an empty array

hence when low > ~~end~~ high → return

∴ if(low ≥ high) return;

right subarray → [s to high]
quickSort(arr, start, high) low high
as parameters we take as (arr, low, high)

5th pass {10} l==h

{ } an empty array l==h

In the last you'll get

[10, 30, 40, 50, 80, 90]

Code:-

```
import java.util.Arrays;
import java.util.Scanner;
public class Main{
    public static void main(String[] args){

int[] arr = {1, 2, 3, 9, 8};
quickSort(arr, 0, arr.length-1);
System.out.println(Arrays.toString(arr));

}
//return type is void because we are not returning anything.
//instead we directly modify the original array in-place.
public static void quickSort(int[] arr, int low, int high)
{ if(low >= high) return; //explained above
  int s = low; //start pointer
  int e = high; // end pointer.
  int m = s + (e-s)/2; //middle index.
  int pivot = arr[m]; //pivot ele. (chosen as middle ele)
//partitioning.
while(s<=e){
  //move s forward until we find an ele. >= pivot
  while(arr[s] < pivot){
    s++;
}
```

```
//move e backward until we find an ele <= pivot.
while (arr[e] > pivot){
    e--;
}
/*
→ Imp note:
→ Even when s>e, while loop may still run one more time
→ As increment/decrement of s & e happens inside the loop.
→ If we directly swapped without checking again, it could
  swap wrongly swap.
→ elements after the partition condn has already broken.
∴ we add the extra check if (s <= e) before swapping
→ to ensure that swapping only happens when pointers are
  valid. (i.e. bot s & pointers are valid only when s<=e) */
    if (s<=e){
        int temp = arr[s];
        arr[s] = arr[e];
        arr[e] = temp;
        s++;
        e--;
    }
} // recursive calls on left&right parts.
quickSort (arr, low, e); // left subarray.
quickSort (arr, s, high); // right subarray.
}
}
```

(Hoare's Partition Scheme) (not Lomuto

7. Recurrence relation:

Let $T(n)$ = time taken to quick sort n elements.

- Quick sort does 2 things:
  1. Partitioning step (linear scan of array) → takes $O(n)$ (movement of s & e & it's comparison with pivot)
  2. Recursive calls on two parts (size depend on pivot.) of 2 partitioned array. more on side / with some elements

* General form:-

$$T(n) = T(k) + T(n-k-1) + O(n)$$

- k = number of elements in left subarray.
- $n-k-1$ = number of elements in right subarray.
- $O(n)$ = time for partioning (i.e. scanning) at each level.

e.g. $[\underbrace{3, 4, 5}_{k}, \underbrace{6, 7}_{n-k-1}]$ (1 for pivot ele.)

n

---

* Best case (Balanced partition) → when pivot is always middle ele. (present in sorted array)

e.g. [1, 5, 2, 6, 3]
pivot = index ⇒ 4
⇒ ele ⇒ 3 (middle ele. as per sorted array)

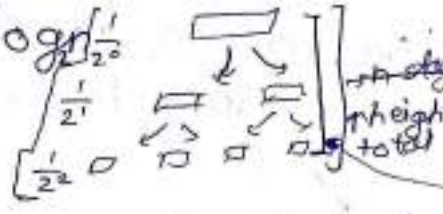
when → Pivot splits array into 2 halves always ($k \approx n/2$) & $(n-k-1) \approx n/2$

→ Recurrence:-

$$T(n) = 2T(n/2) + O(n)$$

(2 → we divide search space by 2 in 2 halves)

→ Height of recurrence tree = $\log_2 n$

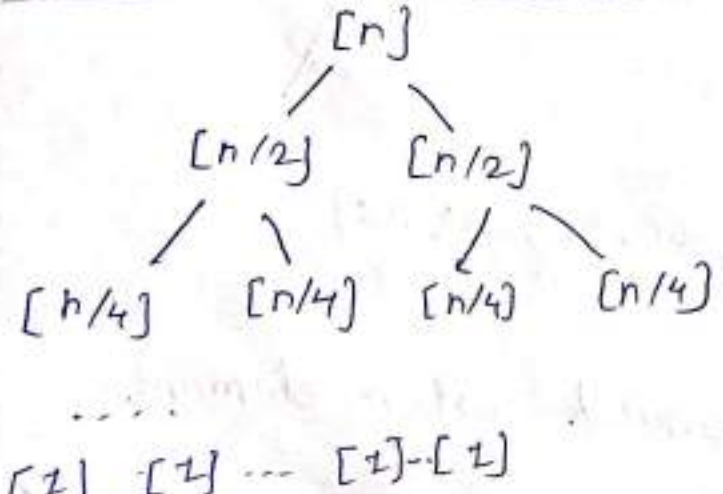


→ work per level = $O(n)$ → scanning at each level.

→ Total time = $O(n \log n)$

* $O(n)$ → no. of steps (work) happen at each stage
* $\log n$ → height of recurrence tree / division tree

⊛ Best case Recursion Tree:



e.g. [40, 10, 90, 50, 70, 30]
Pivot splits nearly equal each step → Balanced tree → $O(n \log n)$

pivot = 40 (during first recursive call) → first ele.

---

* Worst case (unbalanced partition) → when ele. is largest / smallest in a whole array at each step.

→ Pivot always smallest / largest → one side empty, other side has n-1.

→ Recurrence:-

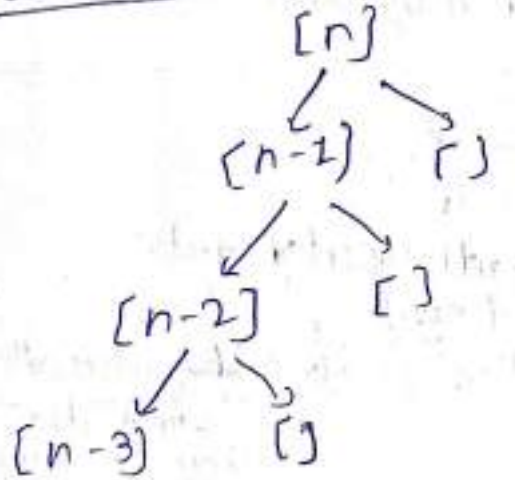
$T(n) = T(n-1) + O(n)$

→ other array/part would be empty hence only (n-1) quick sort recursive call happens on only n-1 elements ~~at each level~~ everytime

→ Expanding:- $O(n^2) + O(n) \approx O(n^2)$

- $T(n) = n + (n-1) + (n-2) + \dots + 1$ → how array size will get reduced everytime.
- $= O(n^2)$

* Worst case Recussion Tree (Sorted I/p):-



eg. [10, 20, 30, 40, 50] (sorted ascending)

→ pivot = first ele. → always smaller

→ skewed tree (chain-like) → $O(n^2)$

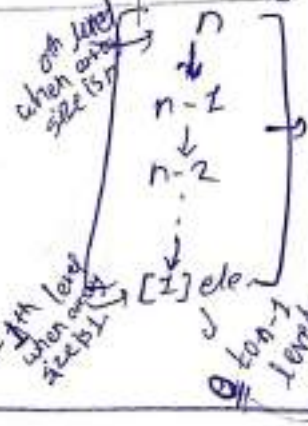
[1] ele. [] → when ele ≤ 1 left in an array, recussion/funct^n ends there & starts going up in a recussive tree.

* Complexity table:-

as no. of recussions = no. of levels = $\log_2 n$ (height of recurrence tree)

case	Time. C	S. C	Note.
① Best	$O(n \log n)$	$O(\log n)$	Balanced pivot (middle ele.)
② Avg.	$O(n \log n)$	$O(\log n)$	Random pivot helps.
③ Worst	$O(n^2)$	$O(n)$	Already sorted i/p.

logn levels will be filled in stack so stack space will increase but no any extra space is used to create a new array as it's In place sorting algo.



→ n levels in worst case. (hence n stack levels are required)

∴ S.C → $O(n)$ due to stack space.

but no space is utilized to create any new array

→ Quick sort features:

① Inplace (no new array is created)
② Non-stable / unstable (order of indices of ele's changes after sorting)

③ MS. is better in linked list due to memory allocation
(Merge sort)
→ not continuous (as in linked list ele. are not continuous, or Linked list are not continuous memory allocation (like arrays), they have random memory allocation.

---

* Hybrid Sorting Algorithm (Tim sort):-

→ Merge Sort + Insertion sort
(works well with partially sorted data (eg. half sorted etc.))

* do Arrays.Sort(arr) → see documentation of it (~~whi~~ internally which algo. they are using

mostly ~~they~~ we quick sort ~~along~~ ~~with mer~~ ~~inserti~~ & Or Tim sort depending on a language